



LET'S START — DAY 1 (FOUNDATION)



Topic 1: What is Spring Framework?

👉 Spring is a **Java framework** that helps you build applications easily by handling:

- Object creation
 - Dependency management
 - Boilerplate code
-



Real-World Analogy

Without Spring:

You cook everything yourself (gas, ingredients, utensils)

With Spring:

You go to a **restaurant** → everything is prepared for you 🍴

Spring manages everything → you only focus on business logic.



Topic 2: Inversion of Control (IOC)



Without IOC (Traditional Java)

Java

```
class Engine {  
    void start() {  
        System.out.println("Engine started");  
    }  
}
```

```
class Car {
```

```
Engine engine = new Engine(); // tightly coupled
```

```
void drive() {  
    engine.start();  
}  
}
```

Show more lines

👉 Problem:

- **Car** controls **Engine** creation → tightly coupled
-

✅ With IOC (Spring Concept)

Java

```
class Car {  
    Engine engine;
```

```
    Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

Show more lines

👉 Now:

- Spring creates objects ✅
 - Spring injects dependencies ✅
-

💡 This is called: 👉 **Inversion of Control = Spring controls objects, not you**

✅ Topic 3: Dependency Injection (DI)

👉 DI = Process of injecting required objects

Types:

1.  Constructor Injection  (BEST)
 2. Setter Injection
 3. Field Injection (avoid)
-

Example:

Java

@Component

class Engine {

public void start() {

System.out.println("Engine started");

}

}

@Component

class Car {

private Engine engine;

@Autowired

public Car(Engine engine) {

this.engine = engine;

}

public void drive() {

engine.start();

}

}

Show more lines

Key Annotations (IMPORTANT)

Annotation	Meaning
------------	---------

@Component	Create Bean
@Autowired	Inject Dependency
@SpringBootApplication	Main class

INTERVIEW QUESTIONS

1. What is IOC?
 2. What is Dependency Injection?
 3. Difference between IOC and DI?
 4. Why constructor injection is preferred?
-

MINI TASK (DO THIS)

Try to:

1. Create `Laptop` and `Processor` classes
 2. Inject Processor into Laptop using constructor
 3. Print: "Processor running"
-

DAY 2: Spring Boot Setup + First Application + REST API

Goal of Day 2

By the end of today, you will:  Understand Spring Boot architecture
 Create your first app

- ✓ Run server
 - ✓ Build your first REST API
-

✓ PART 1: What is Spring Boot?

👉 Spring Boot = **Simplified version of Spring**

✗ Problem in Spring (Old way)

- Too much configuration (XML)
 - Manual setup
 - Server setup required
-

✓ Spring Boot Solution

- Auto configuration ✓
 - Embedded server ✓ (Tomcat)
 - Minimal setup ✓
-

🧠 Real-World Analogy

Spring → You build car parts manually

Spring Boot → You get a **ready-made car** 🚗

✓ PART 2: Spring Boot Project Structure

When you create project, you'll see:

None

src/main/java

```
└─ com.example.demo
    ├── DemoApplication.java
    ├── controller
    ├── service
    └── repository
```

src/main/resources

```
└─ application.properties
```

Important File

Main Class

Java

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Show more lines



What does **@SpringBootApplication** do?

It combines:

Java

@Configuration

@EnableAutoConfiguration

@ComponentScan

Show more lines

✓ PART 3: Create First Spring Boot Project

👉 Steps:

1. Go to 👉 <https://start.spring.io>
 2. Select:
 - Project: Maven
 - Language: Java
 - Dependency: ✓ Spring Web
 3. Click **Generate**
 4. Import into IntelliJ/Eclipse
-

✓ PART 4: Run Application

Run main class:

👉 You'll see:

None

Tomcat started on port 8080

✓ Your server is running!

✓ PART 5: Create Your First REST API 🔥

✅ Step 1: Create Controller

Java

```
package com.example.demo.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class HelloController {
```

```
@GetMapping("/hello")
```

```
public String sayHello() {
```

```
    return "Hello Sameer! Welcome to Spring Boot 🚀";
```

```
}
```

```
}
```

Show more lines

✅ Step 2: Run and Test

Open browser or Postman:

None

<http://localhost:8080/hello>

👉 Output:

None

Hello Sameer! Welcome to Spring Boot 🚀

✅ PART 6: Important Annotations (MUST KNOW)

Annotation	Use
<code>@RestController</code>	Marks class as API controller
<code>@GetMapping</code>	Handle GET requests
<code>@PostMapping</code>	Handle POST requests
<code>@RequestMapping</code>	Base URL

PART 7: Request Types

Method	Purpose
GET	Fetch data
POST	Create data
PUT	Update
DELETE	Remove

PART 8: Add Path Variable

Java

```
@GetMapping("/hello/{name}")
public String greet(@PathVariable String name) {
    return "Hello " + name;
}
```

Show more lines

👉 Test:

None

/hello/Sameer

✓ PART 9: JSON Response (Important for APIs)

Java

```
@GetMapping("/user")
public User getUser() {
    return new User("Sameer", 25);
}
```

Show more lines

✓ User Class

Java

```
class User {
    String name;
    int age;

    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Show more lines

👉 Output:

JSON

```
{
  "name": "Sameer",
  "age": 25
}
```

Show more lines



INTERVIEW QUESTIONS

1. What is Spring Boot?
 2. Difference: Spring vs Spring Boot?
 3. What is `@SpringBootApplication`?
 4. What is embedded server?
 5. What is `@RestController`?
-



MINI TASK (DO THIS)

Try coding these:

Task 1:

Create API:

```
None  
/welcome
```

Return:

```
None  
Welcome to your first API
```

Task 2:

Create API:

None

```
/add/{a}/{b}
```

Return sum

Task 3:

Create User API:

- Return JSON with name + role

DAY 3: POST API + RequestBody + DTO + Layered Architecture

Goal of Today

By end of Day 3 you will: ☒ Create POST API

☒ Accept JSON request

☒ Understand DTO

☒ Learn real-world architecture (Controller → Service → Repository)

PART 1: What is POST API?

👉 POST = Used to **send data to server**

Example:

Client sends:

JSON

```
{  
"name": "Sameer",  
"role": "Developer"  
}
```

Show more lines

Server stores/processes it 

PART 2: **@RequestBody** (VERY IMPORTANT)

 Converts JSON → Java Object

Example

```
Java  
@PostMapping("/user")  
public String createUser(@RequestBody User user) {  
    return "User created: " + user.getName();  
}
```

Show more lines

User Class

```
Java  
class User {  
    private String name;  
    private String role;  
  
    // getters & setters  
}
```

..

Show more lines

Input (Postman)

POST → <http://localhost:8080/user>

JSON

```
{  
  "name": "Sameer",  
  "role": "Backend Developer"  
}
```

Show more lines

Output

None

User created: Sameer

PART 3: Why NOT use Entity directly? (Enter DTO 🔥)

👉 In real projects, we don't expose entity directly

Analogy

Entity = Bank database table

DTO = What you show to customer

👉 Never expose internal structure ❌

✓ DTO Example

Java

```
class UserDTO {  
    private String name;  
    private String role;
```

```
// getters & setters  
}
```

Show more lines

✓ Controller using DTO

Java

```
@PostMapping("/user")  
public String createUser(@RequestBody UserDTO userDTO) {  
    return "User created: " + userDTO.getName();  
}
```

Show more lines

✓ PART 4: Layered Architecture (MOST IMPORTANT 🔥 🔥 🔥)

👉 This is what makes you **job-ready**

✓ Structure

None

Controller → Service → Repository → Database

Simple Understanding:

Layer	Responsibility
Controller	Handles HTTP request
Service	Business logic
Repository	DB interaction

PART 5: Build Proper Structure

1. Controller

Java

@RestController

```
public class UserController {
```

```
    private final UserService userService;
```

```
    public UserController(UserService userService) {
```

```
        this.userService = userService;
```

```
    }
```

```
    @PostMapping("/user")
```

```
    public String createUser(@RequestBody UserDTO userDTO) {
```

```
        return userService.createUser(userDTO);
```

```
    }
```

```
}
```

Show more lines

✓ 2. Service

Java

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
    public String createUser(UserDTO userDTO) {
```

```
        return "User saved: " + userDTO.getName();
```

```
    }
```

```
}
```

Show more lines

✓ Key Annotation

Annotation	Meaning
@Service	Marks business logic layer

✓ PART 6: Dependency Injection Used Here

👉 Spring automatically injects:

Java

```
public UserController(UserService userService)
```

Show more lines

✓ This is **constructor injection (BEST PRACTICE)**



PART 7: How Data Flows

None

Client → Controller → Service → Response

Example:

None

POST /user

↓

Controller receives JSON

↓

Converts to DTO

↓

Calls Service

↓

Service processes

↓

Response return



PART 8: Add Validation (BONUS)

Java

```
import jakarta.validation.constraints.NotBlank;
```

```
class UserDTO {
```

```
@NotBlank
```

```
private String name;
```

```
private String role;
```

```
}
```

Show more lines

Controller:

Java

```
@PostMapping("/user")
public String createUser(@Valid @RequestBody UserDTO userDTO) {
    return userService.createUser(userDTO);
}
```

Show more lines



INTERVIEW QUESTIONS

1. What is `@RequestBody`?
 2. Difference between DTO and Entity?
 3. What is layered architecture?
 4. Why use Service layer?
 5. What is constructor injection?
-



MINI TASK (VERY IMPORTANT)

Do this HANDS-ON:



Task 1:

Create POST API:

```
None
/employee
```

Input:

JSON

```
{
```

```
"name": "Sameer",  
"department": "IT"  
}
```

Show more lines

Return:

None

Employee added: Sameer

Task 2:

Implement: ☒ Controller

☒ Service

☒ DTO

Task 3 (Challenge):

Add:

None

GET /employee/{name}

Return employee data

DAY 4: FULL CRUD APIs (GET, POST, PUT, DELETE)

Goal of Day 4

By the end of today, you will:  Build full CRUD APIs

 Store data (in-memory for now)

 Understand real API design

PART 1: What is CRUD?

Operation	HTTP Method	Meaning
Create	POST	Add data
Read	GET	Fetch data
Update	PUT	Modify data
Delete	DELETE	Remove data

PART 2: We Will Build

 Employee Management API

No database yet (we use List) 

PART 3: Project Structure

None

controller

service

dto



PART 4: Employee DTO

Java

```
public class EmployeeDTO {
```

```
    private int id;
```

```
    private String name;
```

```
    private String department;
```

```
    // getters & setters
```

```
}
```

Show more lines



PART 5: Service Layer (CORE LOGIC)

Java

```
import org.springframework.stereotype.Service;
```

```
import java.util.*;
```

```
@Service
```

```
public class EmployeeService {
```

```
    private List<EmployeeDTO> employees = new ArrayList<>();
```

```
    // CREATE
```

```
    public String addEmployee(EmployeeDTO emp) {
```

```
        employees.add(emp);
```

```
        return "Employee added: " + emp.getName();
```

```
    }
```

```
    // READ ALL
```

```
    public List<EmployeeDTO> getAllEmployees() {
```

```
        return employees;
```

```
    }
```

```
// READ BY ID
public EmployeeDTO getEmployeeById(int id) {
    return employees.stream()
        .filter(e -> e.getId() == id)
        .findFirst()
        .orElse(null);
}

// UPDATE
public String updateEmployee(int id, EmployeeDTO updatedEmp) {
    for (EmployeeDTO emp : employees) {
        if (emp.getId() == id) {
            emp.setName(updatedEmp.getName());
            emp.setDepartment(updatedEmp.getDepartment());
            return "Employee updated";
        }
    }
    return "Employee not found";
}

// DELETE
public String deleteEmployee(int id) {
    boolean removed = employees.removeIf(e -> e.getId() == id);
    return removed ? "Deleted successfully" : "Employee not found";
}
}

Show more lines
```



PART 6: Controller Layer

Java

```
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/employee")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    // CREATE
    @PostMapping
    public String addEmployee(@RequestBody EmployeeDTO emp) {
        return service.addEmployee(emp);
    }

    // READ ALL
    @GetMapping
    public List<EmployeeDTO> getAll() {
        return service.getAllEmployees();
    }

    // READ BY ID
    @GetMapping("/{id}")
    public EmployeeDTO getById(@PathVariable int id) {
        return service.getEmployeeById(id);
    }

    // UPDATE
    @PutMapping("/{id}")
    public String update(@PathVariable int id, @RequestBody EmployeeDTO emp) {
        return service.updateEmployee(id, emp);
    }

    // DELETE
    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id) {
        return service.deleteEmployee(id);
    }
}
```



```
}  
}
```

Show more lines

PART 7: API TESTING (IMPORTANT)

1. CREATE

POST `/employee`

JSON

```
{  
  "id": 1,  
  "name": "Sameer",  
  "department": "IT"  
}
```

Show more lines

2. GET ALL

GET `/employee`

3. GET BY ID

GET `/employee/1`

4. UPDATE

PUT /employee/1

JSON

```
{  
  "name": "Sameer Updated",  
  "department": "Backend"  
}
```

Show more lines

5. DELETE

DELETE /employee/1

PART 8: Important Learnings

👉 You now understand:

- Full API lifecycle 
 - Layered architecture 
 - Data flow 
-

PART 9: Problem with Current Approach



-  Data lost when server restarts
-  Not scalable
-  No real DB

👉 Solution = **Database (Day 5 )**



INTERVIEW QUESTIONS

1. What is CRUD?
 2. Difference between PUT and POST?
 3. How does DELETE work internally?
 4. Why use Service layer?
 5. What happens if id not found?
-



MINI TASK (IMPORTANT)



Task 1:

Add field:

None

salary



Task 2:

Create API:

None

GET /employee/department/{dept}



Task 3 (Challenge 🔥🔥):

Return:

None

Total number of employees

API:

None

GET /employee/count

DAY 5: DATABASE INTEGRATION (Spring Data JPA + Hibernate)

Goal of Today

By end of today, you will: ☒ Connect Spring Boot with database

☒ Store real data

☒ Learn JPA + Hibernate

☒ Replace List with Database

PART 1: What is JPA & Hibernate?

JPA (Java Persistence API)

 Specification (rules)

Hibernate

 Implementation of JPA

Simple Understanding

Concept	Meaning
JPA	Interface
Hibernate	Actual engine

Real-Life Analogy

JPA = Car blueprint 🚗

Hibernate = Actual car engine ⚙️

PART 2: Add Dependencies

In `pom.xml`:

XML

```
<dependencies>
```

```
<!-- Spring Web -->
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<!-- JPA -->
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
</dependency>
```

```
<!-- Database (H2 or MySQL) -->
```

```
<dependency>
```

```
<groupId>com.h2database</groupId>
```

```
<artifactId>h2</artifactId>
```

```
</dependency>
```

```
</dependencies>
```

Show more lines

✓ PART 3: Configure Database

👉 application.properties

✓ For H2 (quick start)

```
.properties
```

```
spring.datasource.url=jdbc:h2:mem:testdb
```

```
spring.datasource.driverClassName=org.h2.Driver
```

```
spring.datasource.username=sa
```

```
spring.datasource.password=
```

```
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

```
spring.h2.console.enabled=true
```

```
spring.jpa.hibernate.ddl-auto=update
```

Show more lines

✓ Run app → open:

None

<http://localhost:8080/h2-console>

✓ PART 4: Create ENTITY (MOST IMPORTANT 🔥)

👉 Replace DTO with ENTITY for DB mapping

✅ Employee Entity

Java

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "employees")
```

```
public class Employee {
```

```
@Id
```

```
private int id;
```

```
private String name;
```

```
private String department;
```

```
private double salary;
```

```
// getters & setters
```

```
}
```

Show more lines

🔥 Important Annotations

Annotation	Meaning
@Entity	Map class → DB table
@Table	Table name
@Id	Primary key

✅ PART 5: Repository Layer

👉 Spring will auto-implement it 🤖

Java

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
}
```

Show more lines

✅ What you get FREE:

- ✅ save()
 - ✅ findAll()
 - ✅ findById()
 - ✅ deleteById()
-

✅ PART 6: Service Layer (Updated)

Java

```
import org.springframework.stereotype.Service;  
import java.util.List;
```

```
@Service
```

```
public class EmployeeService {
```

```
    private final EmployeeRepository repo;
```

```
    public EmployeeService(EmployeeRepository repo) {  
        this.repo = repo;  
    }
```

```
// CREATE
```

```
    public Employee addEmployee(Employee emp) {  
        return repo.save(emp);  
    }
```



```
}
```

```
// READ ALL
```

```
public List<Employee> getAll() {  
    return repo.findAll();  
}
```

```
// READ BY ID
```

```
public Employee getById(int id) {  
    return repo.findById(id).orElse(null);  
}
```

```
// DELETE
```

```
public String delete(int id) {  
    repo.deleteById(id);  
    return "Deleted";  
}  
}
```

Show more lines



PART 7: Controller Layer (Updated)

Java

```
import org.springframework.web.bind.annotation.*;  
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/employee")
```

```
public class EmployeeController {
```

```
    private final EmployeeService service;
```

```
    public EmployeeController(EmployeeService service) {  
        this.service = service;  
    }
```

```
@PostMapping
public Employee add(@RequestBody Employee emp) {
    return service.addEmployee(emp);
}
```

```
@GetMapping
public List<Employee> getAll() {
    return service.getAll();
}
```

```
@GetMapping("/{id}")
public Employee getByid(@PathVariable int id) {
    return service.getByid(id);
}
```

```
@DeleteMapping("/{id}")
public String delete(@PathVariable int id) {
    return service.delete(id);
}
}
```

Show more lines



PART 8: How Data Flows NOW 🔥

None

Client → Controller → Service → Repository → Database



PART 9: Test APIs

CREATE

POST `/employee`

JSON

```
{  
  "id": 1,  
  "name": "Sameer",  
  "department": "IT",  
  "salary": 50000  
}
```

Show more lines

GET ALL

GET `/employee`

DELETE

DELETE `/employee/1`

PART 10: H2 Console Check

Run query:

SQL

```
SELECT * FROM employees;
```

Show more lines

 You'll see real stored data

✓ PART 11: AUTO ID GENERATION (IMPORTANT)

Java

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private int id;

Show more lines

👉 DB will generate ID automatically ✓

✓ PART 12: Common Interview Questions



1. What is JPA?
 2. Difference: JPA vs Hibernate?
 3. What is Entity?
 4. What is JpaRepository?
 5. What is @Id?
 6. What is ddl-auto?
-

✓ MINI TASK (VERY IMPORTANT)

🔥 Task 1:

Use @GeneratedValue instead of manual ID

Task 2:

Create API:

None

```
GET /employee/name/{name}
```

(Hint: custom method in Repository)

Task 3 (Challenge):

Add:

None

```
salary > 50000 filter
```

DAY 6: RELATIONSHIPS + CUSTOM QUERIES + PAGINATION

Goal of Today

By the end of today, you will: ☒ Understand DB relationships (MOST IMPORTANT  )

- ☒ Implement OneToMany, ManyToOne
 - ☒ Write custom queries
 - ☒ Use pagination
-

PART 1: Why Relationships?

👉 Real-world example:

None

One Department → Many Employees

👉 So tables are linked:

None

Department (1) ----- (Many) Employee

🧠 Real-Life Analogy

Department = Classroom

Employees = Students

👉 One classroom → many students ✓

✓ PART 2: Entity Relationship Types

Type	Meaning
OneToOne	1 → 1
OneToMany	1 → many ✓
ManyToOne	many → 1 ✓
ManyToMany	many ↔ many

✓ PART 3: Create Department Entity

Java

```
import jakarta.persistence.*;
import java.util.List;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees;

    // getters & setters
}
Show more lines
```



PART 4: Update Employee Entity

Java

```
import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
    private double salary;

    @ManyToOne
```

```
@JoinColumn(name = "department_id")
private Department department;
```

```
// getters & setters
```

```
}
```

Show more lines

✓ PART 5: Understand Mapping 🔥

👉 In DB:

employee table

None

```
id | name | salary | department_id
```

👉 `department_id` links to Department ✓

✓ PART 6: Repository Layer

Java

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

Show more lines

Java

```
public interface DepartmentRepository extends JpaRepository<Department, Integer> {
}
```

Show more lines

✓ **PART 7: Custom Query Methods** 🔥

👉 Spring auto-generates queries 🤖

✓ **Find by Name**

Java

```
List<Employee> findByName(String name);
```

Show more lines

✓ **Find by Salary > value**

Java

```
List<Employee> findBySalaryGreaterThan(double salary);
```

Show more lines

✓ **Find by Department**

Java

```
List<Employee> findByDepartmentName(String deptName);
```

Show more lines

✓ **PART 8: @Query (Advanced)** 🔥

👉 Write your own SQL/JPQL

Java

```
@Query("SELECT e FROM Employee e WHERE e.salary > :salary")
```

```
List<Employee> getHighSalaryEmployees(double salary);
```

Show more lines

✅ PART 9: Pagination 🔥 (VERY IMPORTANT)

👉 Used in big applications

✅ Example:

Java

```
import org.springframework.data.domain.*;

public Page<Employee> getPaginated(int page, int size) {
    Pageable pageable = PageRequest.of(page, size);
    return repo.findAll(pageable);
}
```

Show more lines

✅ Controller API

Java

```
@GetMapping("/page")
public Page<Employee> getPage(
    @RequestParam int page,
    @RequestParam int size) {

    return service.getPaginated(page, size);
}
```

Show more lines

✅ Call API

None

`/employee/page?page=0&size=5`

✓ PART 10: Data Flow with Relationships

None

`Client → Controller → Service → Repository → DB (JOIN happens)`

✓ PART 11: Common Mistakes ⚠

✗ Infinite JSON loop

👉 Fix:

Java

```
import com.fasterxml.jackson.annotation.JsonIgnore;
```

```
@OneToMany(mappedBy = "department")
```

```
@JsonIgnore
```

```
private List<Employee> employees;
```

Show more lines

✓ PART 12: Important Interview Questions



1. Difference between OneToMany and ManyToOne?
2. What is `mappedBy`?
3. What is `@JoinColumn`?
4. What is CascadeType?
5. What is Lazy vs Eager loading?

6. How does pagination work?



MINI TASK (CRITICAL FOR YOU)



Task 1:

Create:

None

POST /department

POST /employee (with department)



Task 2:

Fetch:

None

GET /employee/department/{name}



Task 3 (Advanced 🔥🔥):

Return:

None

Employees with salary > 50000 using @Query



Task 4 (PRO LEVEL):

Implement pagination API 

DAY 7: EXCEPTION HANDLING + VALIDATION + CLEAN API DESIGN

Goal of Today

By end of today, you will:  Handle errors professionally

 Validate user input

 Create clean, production-level APIs

PART 1: Problem Without Exception Handling

 Example:

None

GET /employee/100

If ID not found →  returns NULL / error

 This looks unprofessional 

PART 2: Create Custom Exception

Java

```
public class ResourceNotFoundException extends RuntimeException {
```

```
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

Show more lines

✓ **PART 3: Use Exception in Service**

Java

```
public Employee getById(int id) {  
    return repo.findById(id)  
        .orElseThrow(() -> new ResourceNotFoundException("Employee not found with id: " + id));  
}
```

Show more lines

✓ Now: Instead of null → proper error ✓

✓ **PART 4: Global Exception Handler** 🔥 🔥 **(VERY IMPORTANT)**

👉 Handles all exceptions in one place

Java

```
import org.springframework.web.bind.annotation.*;
```

```
@RestControllerAdvice
```

```
public class GlobalExceptionHandler {
```

```
@ExceptionHandler(ResourceNotFoundException.class)
public String handleNotFound(ResourceNotFoundException ex) {
    return ex.getMessage();
}
```

```
@ExceptionHandler(Exception.class)
public String handleGeneric(Exception ex) {
    return "Something went wrong";
}
}
```

Show more lines

✅ Now all errors are handled centrally ✅

✅ PART 5: Better API Response (PRO LEVEL 🔥)

👉 Instead of plain string:

JSON

Employee not found

Show more lines

👉 Return structured response ✅

✅ Create API Response Class

Java

```
public class ApiResponse {
```

```
    private String message;
```

```
    private int status;
```

```
public ApiResponse(String message, int status) {  
    this.message = message;  
    this.status = status;  
}
```

```
// getters
```

```
}
```

Show more lines

✓ Update Exception Handler

Java

```
@ExceptionHandler(ResourceNotFoundException.class)  
public ApiResponse handleNotFound(ResourceNotFoundException ex) {  
    return new ApiResponse(ex.getMessage(), 404);  
}
```

Show more lines

✓ Response:

JSON

```
{  
    "message": "Employee not found with id: 10",  
    "status": 404  
}
```

Show more lines

✓ PART 6: Validation (SUPER IMPORTANT



👉 Prevent bad data from entering system

DTO with Validation

Java

```
import jakarta.validation.constraints.*;
```

```
public class EmployeeDTO {
```

```
    @NotBlank
```

```
    private String name;
```

```
    @NotBlank
```

```
    private String department;
```

```
    @Min(1000)
```

```
    private double salary;
```

```
}
```

Show more lines

PART 7: Enable Validation

Java

```
@PostMapping
```

```
public Employee add(@Valid @RequestBody EmployeeDTO dto) {
```

```
    return service.addEmployee(dto);
```

```
}
```

Show more lines

What Happens Now:

If invalid input:

JSON

```
{  
"name": "",  
"salary": 100  
}
```

Show more lines

👉 Auto error generated ✅

✅ PART 8: Handle Validation Errors 🔥

Java

```
import org.springframework.web.bind.MethodArgumentNotValidException;
```

```
@ExceptionHandler(MethodArgumentNotValidException.class)  
public ApiResponse handleValidation(MethodArgumentNotValidException ex) {
```

```
    String error = ex.getBindingResult()  
        .getFieldError()  
        .getDefaultMessage();
```

```
    return new ApiResponse(error, 400);  
}
```

Show more lines

✅ Output:

JSON

```
{  
"message": "must not be blank",  
"status": 400  
}
```

Show more lines

✓ PART 9: Use **ResponseEntity**

(INTERVIEW FAVORITE)

Java

```
import org.springframework.http.ResponseEntity;
```

```
@GetMapping("/{id}")
public ResponseEntity<Employee> getById(@PathVariable int id) {
    Employee emp = service.getById(id);
    return ResponseEntity.ok(emp);
}
```

Show more lines

👉 For error:

Java

```
return ResponseEntity.status(404)
    .body(null);
```

Show more lines

✓ PART 10: Final Clean Flow 🔥

None

Request → Controller → Service → Repository

↓

Exception (if any)

↓

Global Handler → Clean Response

✓ PART 11: Interview Questions 🔥 🔥

1. What is `@RestControllerAdvice`?
 2. Difference between `@ControllerAdvice` & `@ExceptionHandler`?
 3. What is validation in Spring Boot?
 4. What is `@Valid`?
 5. Why use `ResponseEntity`?
 6. How do you handle global exceptions?
-



MINI TASK (VERY IMPORTANT)



Task 1:

Add validation:

- Name not blank
 - Salary > 1000
-



Task 2:

Throw exception:

None

If employee not found



Task 3:

Return structured API response 



Task 4 (PRO):

Handle validation errors globally ✓

WHAT YOU HAVE ACHIEVED (BIG)

- ✓ REST APIs
- ✓ CRUD
- ✓ Database
- ✓ Relationships
- ✓ Exception handling
- ✓ Validation

👉 You are now **Job-ready Backend Developer (Intermediate Level)** ✓

NEXT LEVEL (VERY IMPORTANT)

👉 DAY 8 OPTIONS:

- ✓  Spring Security (JWT Authentication)
 - ✓  Build COMPLETE Project (Employee Management System – Pro Level)
 - ✓  Microservices (advanced path)
-



MY RECOMMENDATION (FOR YOU)

Sameer, since you're preparing for interviews:

👉 Next BEST step: ✓ Build **complete project with all concepts combined**

OR

- ✓ Learn Spring Security (very high demand)

DAY 8: SPRING SECURITY + JWT AUTHENTICATION

Goal of Today

By the end of today, you will: ☒ Secure your APIs

☒ Implement login system

☒ Understand JWT token

☒ Protect endpoints (role-based access)

☒ PART 1: What is Spring Security?

 Spring Security = Framework to **secure your application**

What it does:

- Authentication ☒ (Who are you?)
 - Authorization ☒ (What can you access?)
-

Real-Life Analogy

Login to Gmail:

- Enter username/password → Authentication
 - Access inbox → Authorization
-

☒ PART 2: Default Behavior

👉 When you add Spring Security:

None

All APIs become LOCKED 🔒

👉 You must login first

✅ PART 3: Add Dependency

XML

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Show more lines

✅ PART 4: Disable Default Security (Temp for learning)

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .authorizeHttpRequests()
        .requestMatchers("/public/**").permitAll()
        .anyRequest().authenticated();

    return http.build();
}
```

Show more lines

✓ PART 5: What is JWT? 🔥 🔥

👉 JWT = JSON Web Token

👉 Used for **stateless authentication**

🧠 Simple Flow

None

User Login → Server creates token → Client stores token
→ Client sends token in every request → Server validates ✓

✓ Sample JWT Token

None

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

✓ PART 6: JWT Components

None

HEADER.PAYLOAD.SIGNATURE

Part	Meaning
Header	Algorithm
Payload	User data

Signature	Security
-----------	----------



PART 7: JWT Dependency

XML

```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt</artifactId>
<version>0.9.1</version>
</dependency>
```

Show more lines



PART 8: Create JWT Utility Class 🔥

Java

```
import io.jsonwebtoken.*;
import java.util.Date;

public class JwtUtil {

    private String secret = "secretKey";

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60))
            .signWith(SignatureAlgorithm.HS256, secret)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parser()
```

```
.setSigningKey(secret)
.parseClaimsJws(token)
.getBody()
.getSubject();
}
}
```

Show more lines

PART 9: Authentication API

Java

```
@RestController
@RequestMapping("/auth")
public class AuthController {

    private JwtUtil jwtUtil = new JwtUtil();

    @PostMapping("/login")
    public String login(@RequestParam String username) {
        return jwtUtil.generateToken(username);
    }
}
```

Show more lines

Call API:

None

POST /auth/login?username=Sameer

👉 Response:

None

JWT_TOKEN

PART 10: Secure APIs Using Token

Create Filter

Java

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

public class JwtFilter extends GenericFilter {

    private JwtUtil jwtUtil = new JwtUtil();

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        String header = req.getHeader("Authorization");

        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            String username = jwtUtil.extractUsername(token);

            System.out.println("User: " + username);
        }

        chain.doFilter(request, response);
    }
}
```

Show more lines

PART 11: Register Filter

Java

```
http.addFilterBefore(new JwtFilter(), UsernamePasswordAuthenticationFilter.class);
```

Show more lines

✓ **PART 12: Protect API** 🔥

Java

```
http.authorizeHttpRequests()  
.requestMatchers("/auth/**").permitAll()  
.anyRequest().authenticated();
```

Show more lines

👉 Now:

- Login API → Open ✓
 - Other APIs → Need token 🔒
-

✓ **PART 13: How to Call Secured API**

Add header:

None

Authorization: Bearer YOUR_TOKEN

✓ **PART 14: Role-Based Auth (Basic Idea)**

Java

```
.hasRole("ADMIN")  
.hasRole("USER")
```

Show more lines

✓ **PART 15: Final Flow** 🔥

None

Login → Get Token

↓

Send token in every request

↓

Filter validates token

↓

Allow / Deny access

🧠 **INTERVIEW QUESTIONS** 🔥 🔥

1. What is Spring Security?
2. What is JWT?
3. Difference: Session vs JWT?
4. What is authentication vs authorization?
5. How token is validated?
6. What is filter in Spring Security?

✓ **MINI TASK (VERY IMPORTANT)**

🔥 **Task 1:**

Create login API that returns token

Task 2:

Secure:

None
`/employee APIs`

Task 3:

Pass token in header 

Task 4 (PRO):

Print username from token in API

WHAT YOU HAVE ACHIEVED

-  Spring Boot 
-  REST APIs 
-  Database 
-  Relationships 
-  Exception Handling 
-  Validation 
-  Security (JWT) 

  You are now **JOB-READY BACKEND DEVELOPER**